

Teil 4:

Test und Verifikation

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Softwarequalität / Codequalität.

- Verschiedene Ansätze zur Sicherstellung von Softwarequalität **benennen**



- Praktiken zur Sicherstellung von Codequalität **benennen** und **erläutern**



- Metriken für Codequalität **kennen** und **berechnen**



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Coding Standards
4. Code Reviews
5. Code Metriken
6. Standards, Literatur und Referenzen

The bitterness of poor quality remains long
after the sweetness of low price is forgotten

Motivation

Software ist omnipräsent in modernen Geräten!

Bugs auch ...

Bugs im Betrieb können wirklich teuer werden!

⇒ Methoden, um Fehler zu finden und zu verhindern

Ariane 5

- Junferflug 4. Juni, 1996
- Explodiert 40 Sekunden nach Start
- Lange teuerster Software Fehler : 500M USD



Fehlerhafte Integration von (unnötigem) Legacy Code:

- Unbehandelte Exception in floating-point Umwandlung
- Umwandlung von a 64-bit int to a 16-bit signed integer
- Funktioniert für Zahlen kleiner oder gleich 2^{15}
- Größere Zahl $\Rightarrow$ Exception ...

Heartbleed

- Schwachstelle in OpenSSL
- Jeder im Internet konnte Speicher von betroffenen Servern auslesen
- Betraf ca. 17% der als Server die SSL Verbindungen anbieten

*"We attacked ourselves from outside, without leaving a trace.
... we were able steal from ourselves
user names and passwords, instant messages, emails and
business critical documents and communication."*



Eingaben des Clients nicht validiert; Annahmen auf Server Seite:

- Client teilt Server ein geheimes Passwort der Länge k_1 mit
- Clients schicken sog. *heartbeat* Nachrichten und bitten um Antwort der Länge k_2 . OpenSSL soll mit vereinbartem Passwort der Länge $k_1 = k_2$ antworten
- Schwachstelle: Server kopiert Speicher der Länge k_2 ohne Prüfung

MacOS Root Rechte



Security

Pro tip: You can log into macOS High Sierra as root with no password

Apple, this is Windows 95 bad – but there is a workaround to kill the bug

By Shaun Nichols in San Francisco 28 Nov 2017 at 20:15 149 SHARE ▼



Updated A trivial-to-exploit flaw in macOS High Sierra, aka macOS 10.13, allows users to gain admin rights, or log in as root, without a password.

- Anmeldung als 'root' ohne Passwort nach Update
- Alle Administrator Rechte
- November 2017
- Auch erstes Update löst Problem nicht vollständig

Autonome Fahrzeuge

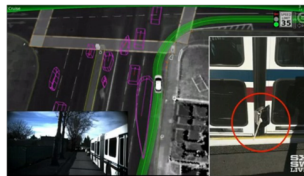
Toyota (Gaspedal klemmt)



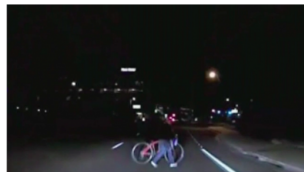
Tesla (LKW nicht erkannt)



Google (Bus gerammt)



Uber (Person überfahren)



Spirit Rover

- Abnormales Verhalten an Sol 18
- Analyse und Reparatur für 11 Sols
- Wieder normales Verhalten ab Sol 33



Flash Dateisystem stürzt ab nach (sehr kurzer) ungetesteter Folge von Operationen

- Kombination von konzeptuellen Fehlern und Konfiguration von Dateisystems und Speicherverwaltung
- Verursacht Folge von Out-of-memory, Neustart, Out-of-memory, ...
- Gelöst durch vorhandenes Notfallsystem

Knight Trading

- 1. August 2012: Bug in Software Update
- 397M Aktien in 45 Minuten gehandelt
- Schwankungen in Aktienkursen von 148 Firmen
- Z.B., Wizzard Software Corporation von \$3.50 auf \$14.76.
- Knight Capital verliert \$440 Mio. (vor Steuern).



Software startet unerwartet Legacy Funktion nach Update:

- Umgenutztes Flag in Kommunikation
- Einer von acht Servern wird vergessen
- Dort: Flag startet Funktion ein, die Algorithmen kalibriert durch das Handeln mit Aktien (gedacht für Simulationsumgebung)

Volkswagen Dieselskandal

Untersuchung / Verfahren laufen immer noch (Folie ist von 2015 ...)

- "Defeat Device" in 11M Dieselfahrzeugen
- Software schaltet Abgasnachbehandlung aus sobald erkannt wird, dass kein Laborversuch gefahren wird
- Scheinbar wurde Funktion heimlich eingebaut

Dienstag, 22. September 2015

Der Hausmeister war's! VW präsentiert Einzeltäter hinter Abgas-Affäre

f 37.7K t + 4.1K



Wolfsburg (dpo) - Im Skandal um manipulierte Abgaswerte von Dieselfahrzeugen setzt VW nach Tagen der Kritik überraschend auf volle Transparenz und legt den alleinigen Verantwortlichen offen. Demnach soll der 61-jährige Hausmeister Hans Böbner seinen Zugang zur Firmenzentrale in Wolfsburg genutzt haben, um eigenhändig Manipulationen an der Software mehrerer Millionen Fahrzeuge vorzunehmen. Inzwischen hat Böbner selbst ein erstes Geständnis abgelegt.

Ausschnitt aus: Der Postillon

- Linux Client löscht Home Verzeichnis (und mehr) bei einigen Nutzern
- Gelöschte Daten können nicht wiederhergestellt werden



pythoner commented on Jan 15, 2015

introduced here Sep 10, 2013 [indrora/steam_latest@ 21cc141](#) line 359

```
rm -rf "$STEAMROOT/**" could be evaluated as rm -rf "/" if $STEAMROOT is empty
```

but what exactly caused this? i've symlinked ~/.local/share/steam to, so i am a bit afraid to start steam :/



TcM1911 commented on Jan 15, 2015

pythoner,

I believe the issue starts on line 19:

```
# figure out the absolute path to the script being run a bit
# non-obvious, the ${0%/*} pulls the path out of $0, cd's into the
# specified directory, then uses $PWD to figure out where that
# directory lives - and all this in a subshell, so we don't affect
# $PWD

STEAMROOT="$(cd "${0%/*}" && echo $PWD)"
STEAMDATA="$STEAMROOT"
```

This probably returns as empty which mean: `rm -rf "$STEAMROOT/**"` is the same as `rm -rf "/"`.

Unsicherer Code: Inhalt von Variable nicht geprüft und Warnungen unterdrückt:

- Variable \$STEAMROOT can leerer String sein
- Programm kann `"rm -Rf /"` ausführen

Qualitätssicherung

- Was ist Qualität von Software?
- Wie kann Qualität sichergestellt werden?
- Warum, Wer, Wann, Was, Wie?

Agenda

1. Motivation
- 2. Begriffe und Definitionen**
3. Coding Standards
4. Code Reviews
5. Code Metriken
6. Standards, Literatur und Referenzen

Definition (Qualität)

Qualität ist der Grad, in dem eine Menge inhärenter Merkmale eines Objekts Anforderungen erfüllt.

Anforderungen (aus Beispielen): Wir wollen, dass unser Produkt ...

- ... nicht explodiert,
- ... Menschen nicht gefährdet,
- ... nicht ausfällt,
- ... Erwartungen der Benutzer erfüllt,
- ... kein Sicherheitsrisiko darstellt.

Wir akzeptieren eine bestimmte Häufigkeit von Problemen
Oft ist die akzeptierte Häufigkeit 0.

Qualitätsmerkmale für Software

- **Funktionalität:**

Korrektheit, Angemessenheit, Interoperabilität, Ordnungsmäßigkeit, Sicherheit

- **Zuverlässigkeit:**

Reife, Fehlertoleranz, Wiederherstellbarkeit • Benutzbarkeit: Verständlichkeit, Bedienbarkeit, Erlernbarkeit, Robustheit

- **Effizienz:**

Wirtschaftlichkeit, Zeitverhalten, Verbrauchsverhalten

- **Wartungsfreundlichkeit:**

Analysierbarkeit, Änderbarkeit, Stabilität, Testbarkeit

- **Übertragbarkeit:**

Anpassbarkeit, Installierbarkeit, Konformität, Austauschbarkeit

Fehler und Ausfälle

Definition (Ausfall)

Wenn eine Anforderung nicht (mehr) erfüllt wird.

Der Moment in dem das System explodiert ...

Definition (Fehler)

Ursache für einen Ausfall

Exception, die geworfen wurde bei 64-bit float nach 16-bit Umwandlung ...

Definition (Qualitätssicherung)

Verhinderung von Ausfällen in Produkten oder Services (durch das Verhindern oder Finden von Fehlern).

■ Produktorientiert:

Software und Zwischenergebnisse werden auf vorher festgelegte Qualitätsmerkmale überprüft.

■ Prozessorientiert:

Methoden, Werkzeuge, Richtlinien und Standards für den Erstellungsprozess der Software

Warum, Wer, Wann, Was, Wie?

Warum?

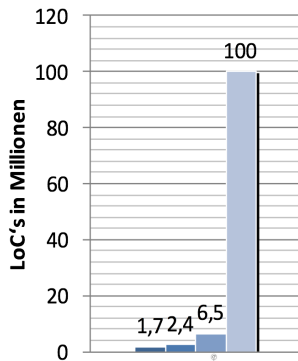
Weil wir gute Menschen sind?

Sondern:

- Um Kunden zufriedenzustellen (damit sie Kunden bleiben)
- Produkthaftung (damit wir nicht verklagt werden)
- Gesetzlich verlangt für bestimmte Produkte (Autos, Flugzeuge, etc.)
- ...

Wer?

Die offensichtlichen Kandidaten: Sicherheitskritische Systeme.



■ F-22 Raptor



F-22

■ F-35 Joint Strike Fighter



787

■ Boeing's 787 Dreamliner

■ PKW
Premiumklasse

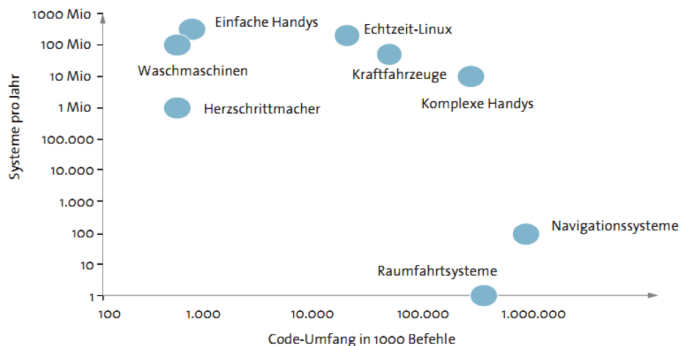


PKW

Wer?

Zwei Gruppen:

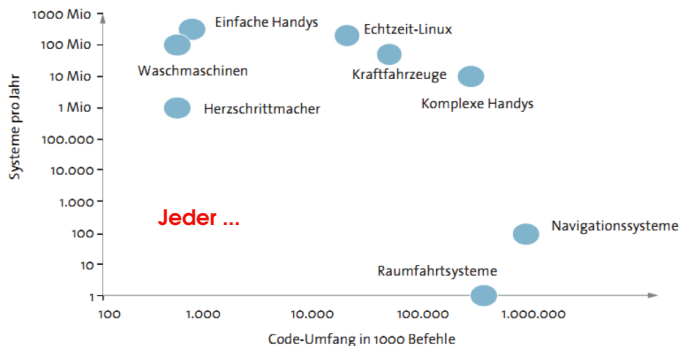
Sicherheitskritische, teure Systems und Systeme mit vielen Installationen.



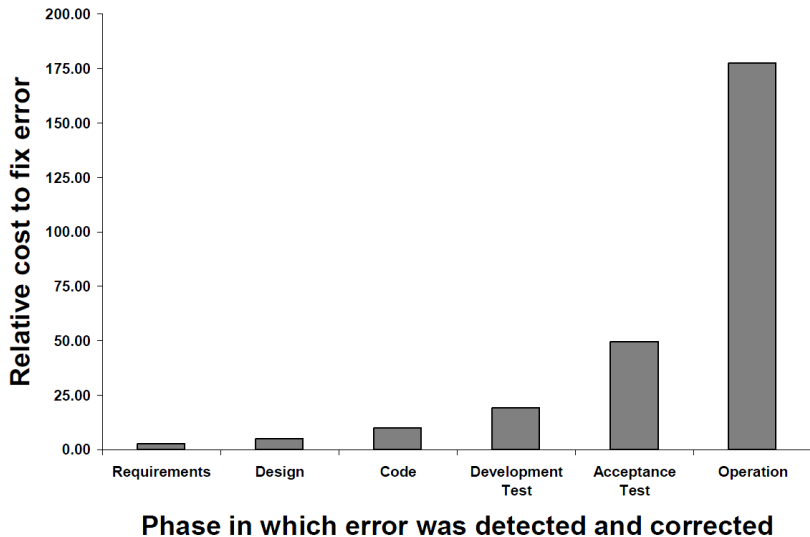
Wer?

Zwei Gruppen:

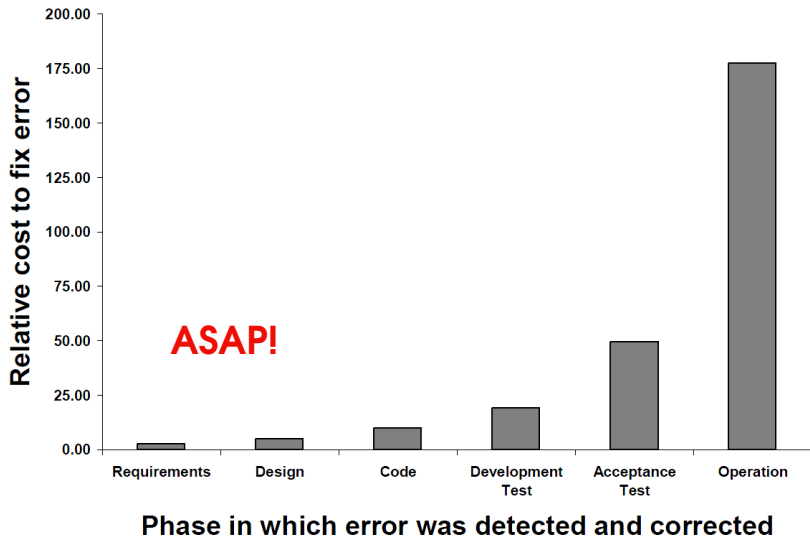
Sicherheitskritische, teure Systems und Systeme mit vielen Installationen.



Wann?



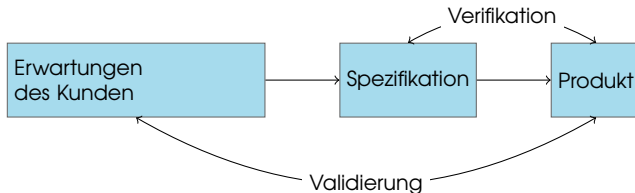
Wann?



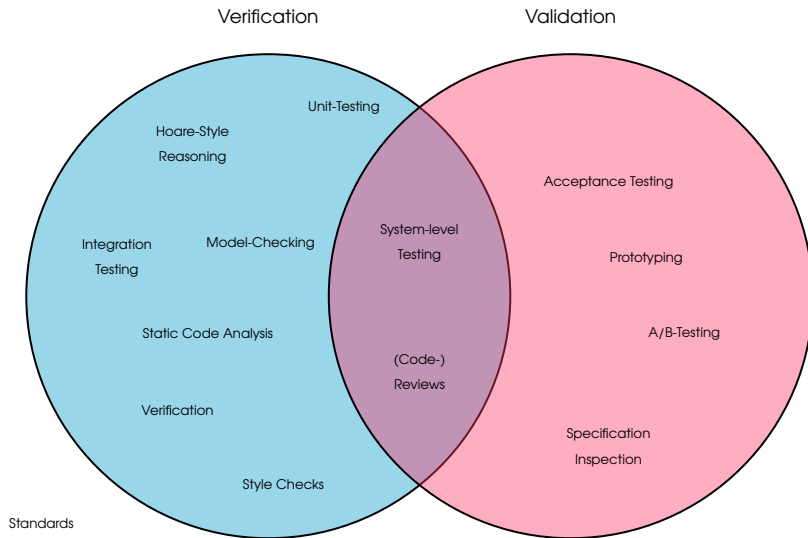
Was?

Verifikation: Bauen wir das Produkt richtig?

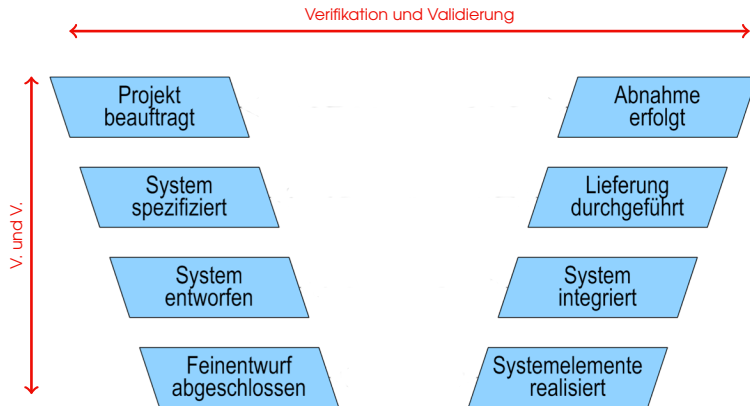
Validierung: Bauen wir das richtige Produkt?



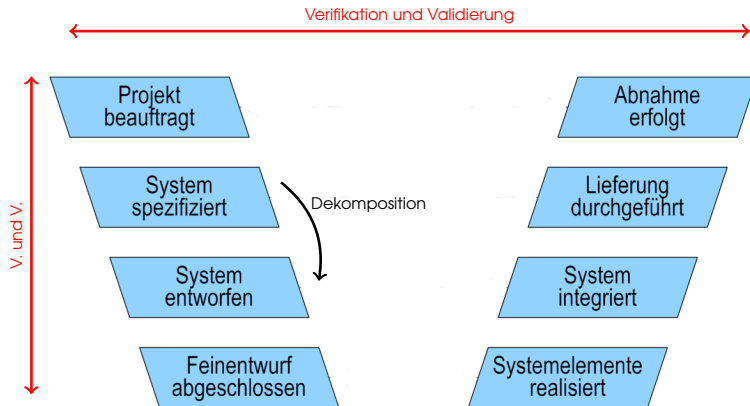
Wie? - Verifikation vs. Validierung



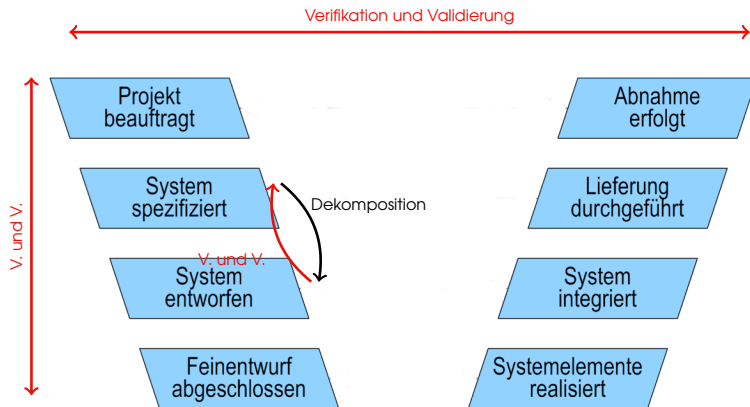
V & V im Entwicklungsprozess



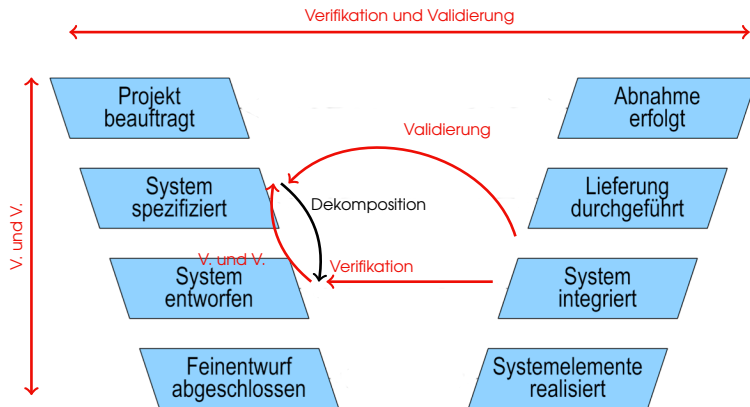
V & V im Entwicklungsprozess



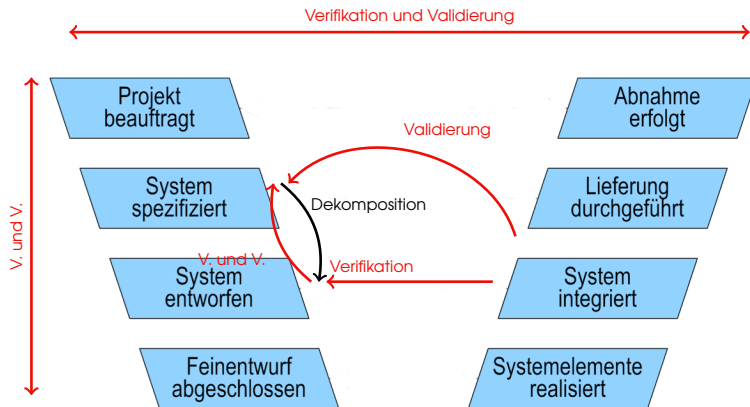
V & V im Entwicklungsprozess



V & V im Entwicklungsprozess



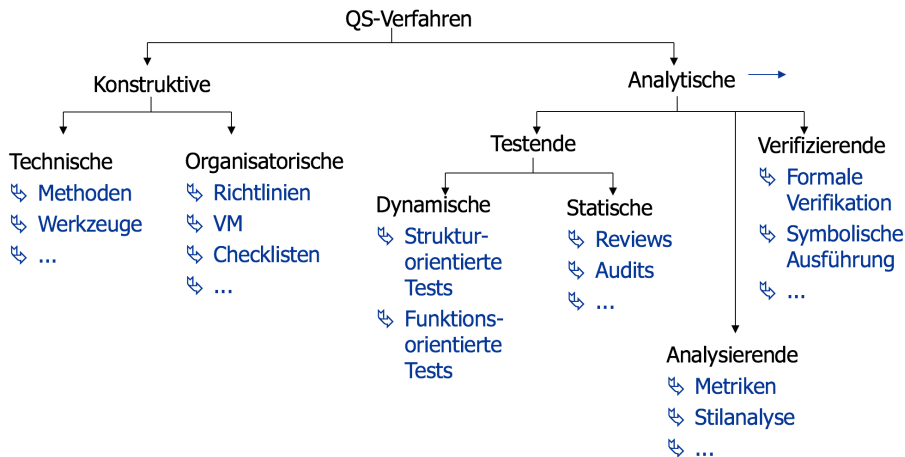
V & V im Entwicklungsprozess



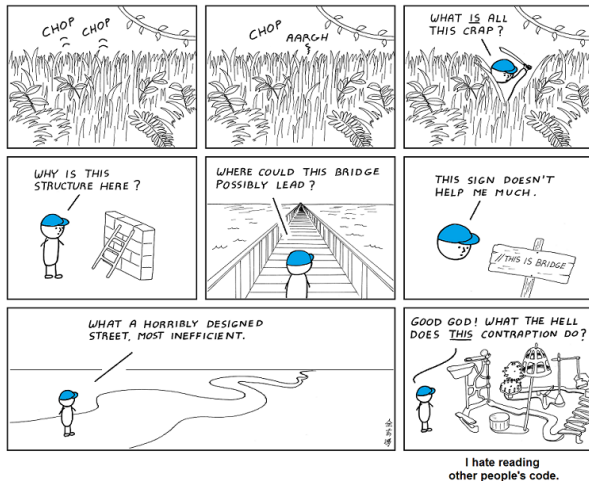
⇒ **Wann?** In jedem Prozessschritt!

Jedes Artefakt des Entwicklungsprozesses kann verifiziert und validiert werden.

Wie? - Klassifizierung von Ansätzen



Code Qualität



Code Qualität: Motivation

In großen Software Projekten ...

- ... Entwickler verteilt über **verschiedene Standorte**,
- ... Viele verschiedene Entwickler, **geteilter Code** (über Zeit bzw. nebeneinander),
- ... **Code aus Jahrzehnten**, auch wenn Entwickler wechseln.

⇒ Entwickler müssen in der Lage sein
den Code von anderen zu lesen und zu verstehen

Code Qualität: Definition

Definition (Code Qualität oder Strukturelle Software Qualität)

Strukturelle Software Qualität ist der Grad, zu dem die Software die nicht-funktionalen Anforderungen erfüllt. Darunter fallen: Zuverlässigkeit / Resilienz, Performanz / Effizienz, Sicherheit, und Wartbarkeit.

Im Gegensatz dazu:

Funktionale Qualität ist der Grad in dem die Software funktionale Anforderungen erfüllt.

Code Qualität: Agenda

Ansätze zur Sicherstellung von Code Qualität:

- ① **Coding Standards** stellen sicher das Code wartbar / lesbar ist und ggf. andere nicht-funktionale Anforderungen erfüllt,
- ② **Code Reviews** stellen gemeinsames Verständnis sicher und finden Fehler möglichst früh,
- ③ **Code Metrics** helfen bei der Bewertung der Qualität von Code.

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Coding Standards
4. Code Reviews
5. Code Metriken
6. Standards, Literatur und Referenzen

Coding Standards



The Ten Commandments, George Bannister, <https://www.flickr.com/>

Beispiele

```
if(foo)
    // bar();
doSomethingElse();
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();
```

Bessere Alternativen

```
if(foo) {
    // bar();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}

boolean b = "".equals(x);
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");

int doIt(int x) {...}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}

boolean b = "".equals(x);
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");

int doIt(int x) {...}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}

boolean b = "".equals(x);

/**
 * computer a random prime
 * number between
 * 0 and upperBound.
 */
int findPrime(int upperBound) {...}
```


Coding Standards

Definition (Coding Standard)

Ein Coding Standard ist eine Menge von Regeln, die einen bestimmten Programmierstil vorschreiben. Regeln beinhalten üblicherweise:

Einrückung, Organisation, Deklarierung, Fehlerbehandlung, Verzweigende Anweisungen, Test Abdeckung, . . .

Beispiele:

- Variablen werden oben in der Klasse deklariert.
- Es soll eine Test-Abdeckung von 80% (Verzweigungen) geben
- Alle Ausnahmen müssen behandelt werden

Coding Standards adressieren Software Qualität

Wartbarkeit

- Dokumentation aller Methoden.
- Klammerung: 1TBS.

Zuverlässigkeit

- Alle Ausnahmen müssen behandelt werden.
- Alle Eingaben müssen validiert werden.

Sicherheit

- SQL Anfragen werden nicht manuell zusammengebaut.
- Code muss getestet sein.

Effizienz

- Nutze `StringBuilder` anstelle von Konkatination.

Coding Standards adressieren Software Qualität

Wartbarkeit

- Dokumentation aller Methoden.
- Klammerung: 1TBS.

Schlechtes Beispiel!
Wie viel?, Wann?, ...



Sicherheit

- SQL Anfragen werden nicht manuell zusammengebaut.
- Code muss getestet sein.

Zuverlässigkeit

- Alle Ausnahmen müssen behandelt werden.
- Alle Eingaben müssen validiert werden.

Effizienz

- Nutze `StringBuilder` anstelle von Konkatination.

Beispiel: NASA JPL

Regeln aus Projekt des Jet Propulsion Laboratory (JPL) für sicherheitskritische Software (für Code in C).

- ① Restrict all code to very simple control flow constructs – do not use goto ...
- ② All loops must have a fixed upper-bound. It must be trivially possible for a checking tool to prove statically that a preset upper-bound on the number of iterations of a loop cannot be exceeded. If the loop-bound cannot be proven statically, the rule is considered violated.
- ③ Do not use dynamic memory allocation after initialization.
- ④ No function should be longer than what can be printed on a single sheet of paper in a standard reference format with one line per statement and one line per declaration. Typically, this means no more than about 60 lines of code per function.
- ⑤ The assertion density of the code should average to a minimum of two assertions per function. Assertions are used to check for anomalous conditions that should never happen in real-life executions. ...
- ⑥ The return value of non-void functions must be checked by each calling function, and the validity of parameters must be checked inside each function.
- ⑦ All code must be compiled, from the first day of development, ... All code must be checked daily with at least one, ..., state-of-the-art static source code analyzer ...
- ⑧ ...

Coding Standards:

Diskussion und Zusammenfassung

Diskussion:

- Regeln müssen spezifisch sein: Z.b. was bedeutet "Don't use Hacks"?
- Weniger ist mehr: Niemand kann 50 Regeln behalten,
- Kontrolle der Einhaltung ist essenziell,
- Automatisierung / Werkzeuge helfen (⇒ Statische Code Analyse).

Zusammenfassung:

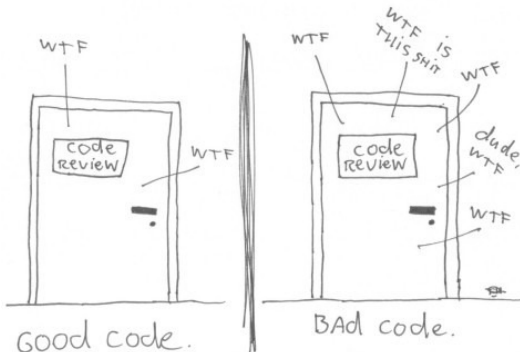
- Regeln / Anweisungen für Format und Eigenschaften von Code
- vgl. "Codex"
- Ziele: Hohe Qualität von Code, keine sogenannten "Code Smells"
- Hohe Qualität: Wartbar, sicher, effizient, ...

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Coding Standards
- 4. Code Reviews**
5. Code Metriken
6. Standards, Literatur und Referenzen

Code Reviews

The ONLY valid measurement
OF code QUALITY: WTFs/minute



(c) 2008 Focus Shift/OSNews/Thom Holwerda - <http://www.osnews.com/comics>

Code Reviews

Definition (Code Review)

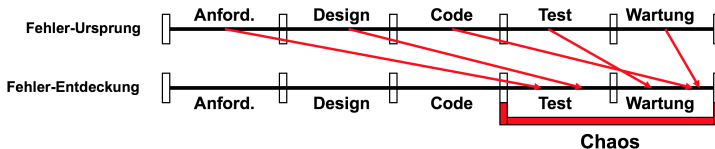
Ein Code Review ist eine systematische Untersuchung und Bewertung von Code — oftmals von anderen Personen als dem Autor des Codes.

Code Reviews helfen dabei ...

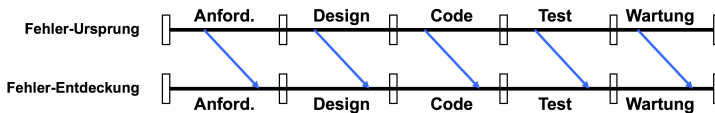
- ... Fehler früh zu finden,
- ... die Qualität des Codes zu erhöhen und sicherzustellen,
- ... geteiltes Verständnis herzustellen.

Code Reviews: Fehler früh finden

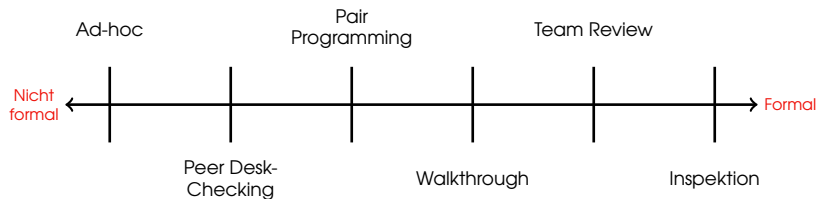
Ohne Reviews



Mit Reviews (ideal)



Code Reviews: Methoden



Code Reviews: Methoden (1)

Ad-hoc

- Kollegen um Hilfe bitten
- Risiko: Nur konkretes Problem wird angeschaut

Peer Desk-Checking

- Reviewer schaut Code am Platz des Kollegen oder auf Papier an

Pair-Programming

- Programmieren zu zweit. Eine Person schreibt, eine prüft.
- Eine Technik des "eXtreme Programming"

Code Reviews: Methoden (2)

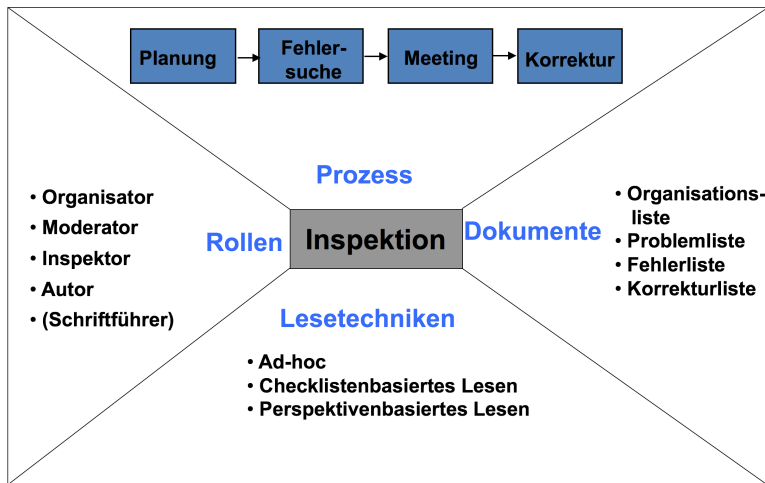
Walkthrough

- Autor präsentiert Code in Termin vor Reviewern
- Kein Vorgeschriebenes Vorgehen
- Risiko: Autor kontrolliert Fokus des Reviews

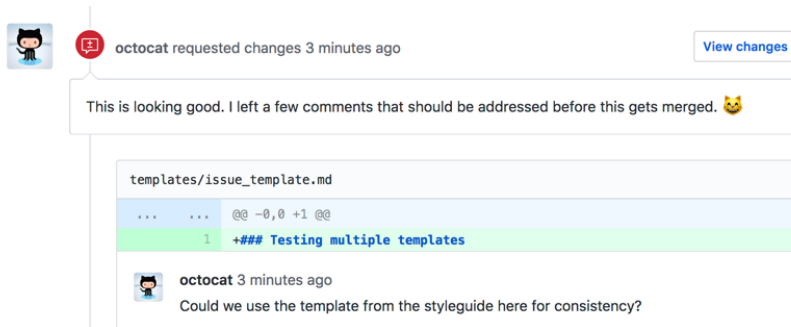
Team-Review

- Mehrere Personen produzieren Review Berichte
- Ergebnisse werden in Termin mit Autor des Codes besprochen

Code Reviews: Inspektion



Beispiel: Git Pull Requests



The screenshot shows a GitHub pull request interface. At the top, the octocat avatar is next to a red circle with a plus sign, followed by the text "octocat requested changes 3 minutes ago". To the right is a blue button labeled "View changes". Below this is a review comment box containing the text "This is looking good. I left a few comments that should be addressed before this gets merged." followed by a cat face emoji. Underneath the comment is a code diff for the file "templates/issue_template.md". The diff shows a line being added, highlighted in green: "1 +### Testing multiple templates". Below the code diff is another comment from octocat, dated "3 minutes ago", asking: "Could we use the template from the styleguide here for consistency?"

octocat requested changes 3 minutes ago [View changes](#)

This is looking good. I left a few comments that should be addressed before this gets merged. 🐱

```
templates/issue_template.md
...    @@ -0,0 +1 @@
1    +### Testing multiple templates
```

octocat 3 minutes ago
Could we use the template from the styleguide here for consistency?

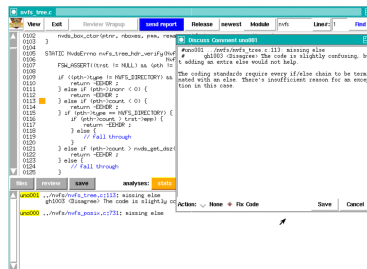
Web-basiertes Review von Pull Requests

- Kommentare möglich an einzelnen Zeilen im Code
- Kommentare müssen adressiert werden

Beispiel: NASA JPL

Werkzeug-gestützte Inspektion:

- Werkzeug für Review / Antworten
- Ergebnisse von automatisierten Analysen im Review
- Diskussion wird gespeichert
- Kurzes Treffen (60-90 min) für jedes Modul
- Kommentare werden im Treffen diskutiert



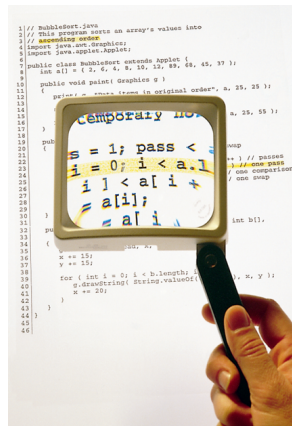
Code Reviews: Zusammenfassung

Ziele:

- Fehler früh finden
- Hohe Code Qualität
- Geteiltes Verständnis
- Besseres Verständnis

Methoden:

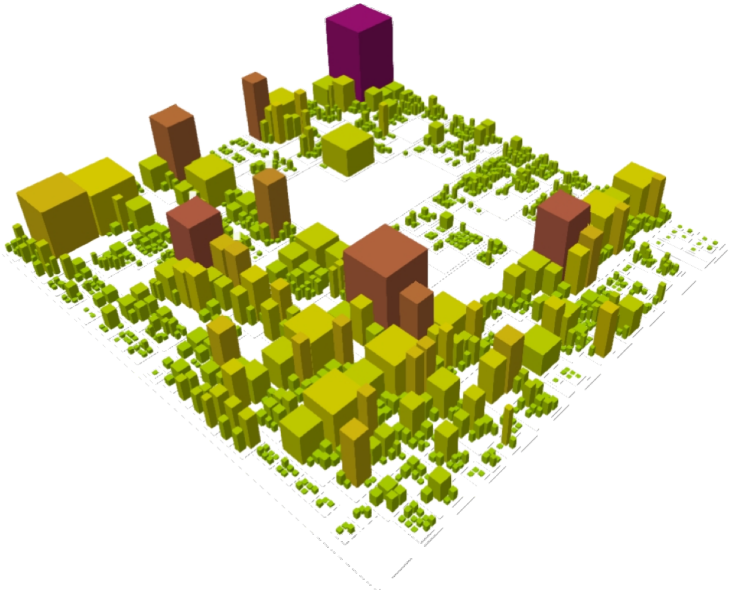
- Von wenig bis sehr formal
- Werkzeugunterstützung
- Ad-hoc, Pair Programming, Inspektion, ...



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Coding Standards
4. Code Reviews
- 5. Code Metriken**
6. Standards, Literatur und Referenzen

Code Metriken



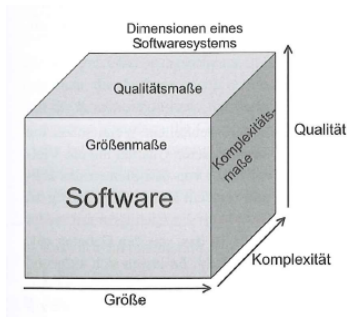
Code Metriken

Definition (Code Metrik)

Eine Code Metrik ist eine Maßeinheit und eine Methode zur Berechnung eines Meßwertes, welcher den Grad ausdrückt, zu welchem Software eine Eigenschaft erfüllt.

Beispiele:

- LOC: Lines of Code
- LCOM: Lack of Cohesion
- CC: Cyclomatic Complexity



Lines of Code (LOC)

Definition (Lines of Code (LOC))

LOC ist die Anzahl der Zeilen im Code einer Software.

- Einfachster Indikator für Komplexität einer Software.
- Ähnliche Metriken: Number of Classes, Number of Methods, Number of ...

Lack of Cohesion Metric (LCOM)

Kohäsion: Innerer Zusammenhang (vgl. Vorlesung Architektur)

Lack of Cohesion Metric (LCOM)

Für eine Klasse mit a Feldern und m Methoden sei $n(A_j)$ die Zahl der Methoden, die auf Feld A_j zugreifen. LCOM ist dann

$$\frac{\left(\frac{1}{a} \sum_{i=1}^a n(A_i)\right) - m}{1 - m}.$$

- Zusammenhang wird geschätzt durch Zugriff
- Hoher LCOM Wert $\Leftrightarrow$ geringe Kohäsion

Beispiel: Lack of Cohesion Metric (LCOM)

Dreieck
-a : int
-b : int
-c : int
+setKannten(a:int, b:int, c:int)
+zeichne()
+umfang()
+flaeche()

Kreis
-r : int
+setRadius(r:int)
+zeichne()
+umfang()
+flaeche()

Dreieck and Kreis:
Jedes Feld wird von jeder Methode genutzt.

Dreieck:

$$a = 3, m = 4, A_1 = 4, A_2 = 4, A_3 = 4$$
$$\Rightarrow LCOM = \frac{(1/3 \cdot 12) - 4}{1 - 4} = 0$$

Kreis:

$$a = 1, m = 4, A_1 = 4$$
$$\Rightarrow LCOM = \frac{(1/1 \cdot 4) - 4}{1 - 4} = 0$$

Figur
-a : int
-b : int
-c : int
-r : int
+zeichneDreieck()
+zeichneKreis()
+flaecheDreieck()
+flaecheKreis()
+umfangDreieck()
+umfangKreis()
+setRadius(r:int)
+setKannten(a:int, b:int, c:int)

Figur = Dreieck + Kreis:

$$a = 4, m = 8, A_1 = 4, A_2 = 4, A_3 = 4, A_4 = 4$$
$$\Rightarrow LCOM = \frac{(1/4 \cdot 16) - 8}{1 - 8} = \frac{4}{7}$$

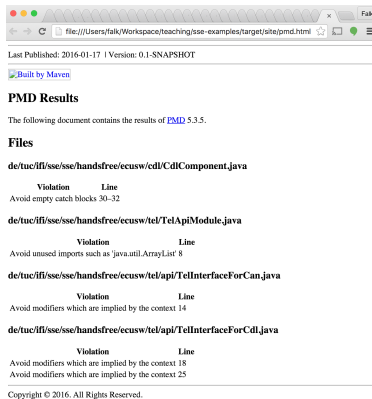
Cyclomatic Complexity (CC)

Im nächsten Foliensatz ...

Beispiel: Werkzeug

PMD sucht in Java Code nach:

- Doppeltem Code
- Komplizierten Ausdrücken
- Vielen anderen Eigenschaften



Wer viel misst, misst Mist

Goal Question Metric Paradigm (GQM)

Goal:

Was ist das Ziel?

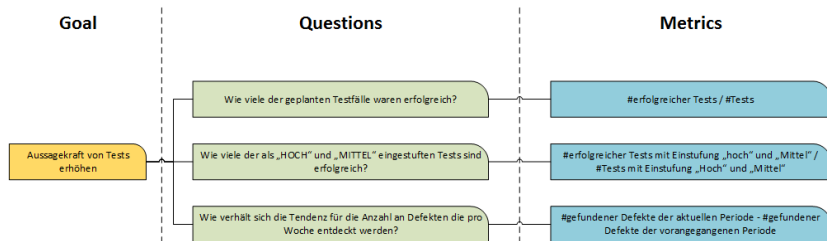
Question:

Was muss dazu analysiert werden?

Metric:

Was ist eine aussagekräftige Metrik?

GQM Beispiel Code Metriken



Code Metriken: Zusammenfassung

Metrik:

- Maßeinheit, Methode zur Bewertung von Code-Qualität
- Für Code oft ausrechenbar
- Beispiele: LOC, LCOM, CC, ...

Ziel:

- Eigenschaft (Qualität) quantifizieren

Vorsicht!

- Messwerte / Metriken sind nur Indikationen
- Zusammenhang Ziel / Frage / Metrik?

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Coding Standards
4. Code Reviews
5. Code Metriken
6. Standards, Literatur und Referenzen

- Dirk W. Hoffmann. Software-Qualität, 2008.
<https://link.springer.com/book/10.1007%2F978-3-540-76323-9>

- **ISO/IEC 25010:2011** Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models
- **ISO/IEC 25000** System und Software-Engineering - Qualitätskriterien und Bewertung von System- und Softwareprodukten (SQuaRE) - Leitfaden für SQuaRE
- **ISO 9126** Software engineering — Product quality was an international standard for the evaluation of software quality (Ersetzt durch ISO/IEC 25000)
- **DIN EN ISO 9000:2015-11** Qualitätsmanagementsysteme – Grundlagen und Begriffe